

## Learning Narrative

The most difficult part of the project was understanding the code itself. Coming from a computer science background, the code structure was not the same, making it difficult to follow compared to a typical computer science programming style. Given the code was underdeveloped, understanding the different functions was initially confusing. To better understand the program, I rewrote the code in a computer science approach to improve the readability of the code as well as to understand the code. While rewriting the code, I also included type hinting to the functions so I could identify the inputs and outputs of each function. After spending time reviewing the code and updating it per the handout, coding the project became the easiest part. The code followed the handout making it manageable given that I have a background in computer science which made understanding the programming logic easier.

The debugging process occurred naturally throughout the development of the code. A common strategy I used was to add print statements in parts of the code to verify that the functions are producing the correct results. However, the most effective debugging method was to compare the output of my program to the figures provided in the handout. By visually comparing the figures it was easy to identify deviations in the code and correct it. When I could not determine the error in the source of my code, I visited office hours to get better feedback on what needed to be fixed. The only AI or LLM tool I used during the process was the built-in autocomplete tool in Visual Studio Code. The tool worked well for repetitive tasks, like writing similar sections of code or updating function that had a similar structure as the previous function. For example, capacity, conduction, and generation functions took in mesh as a parameter, the autocomplete tool was able to autocomplete these functions since they were identical in style with only difference being function name. The autocomplete tool improved efficiency in writing the code as I did not need to repeatedly write the same function. However, at times, the autocomplete tool suggested incorrect parameters and generated functions that did not exist. This resulted in additional debugging issues because of the built-in autocomplete tool.

My attitude toward commercial engineering solvers has not changed since completing this project. I still believe that engineering solvers provide a major advantage when solving repetitive or complex problems. Human error is more likely in repetitive calculation, but engineering solvers eliminate the repetitive task by making the calculation correct. In addition,

engineering solvers are useful in solving complex problems that would be very difficult to do by hand. For example, I work as an intern for an HVAC engineering consulting firm, at the firm we use programs like Trace3D which can give estimated load calculations for a building. Being able to compute the load requirements for an entire building through a engineering solver allows engineers to use their time in a more effective way by focusing on design decisions rather than repetitive tasks.

Overall, I thought the project was well structured because the handout closely matched the code that needed to be implemented. However, there are a few improvements I recommend for next semester. First, I think the handout should include a dedicated section for listing all variable names and symbols that are used throughout the code. As I have stated before, I come from a computer science background where the naming of the variable is incredibly important to understanding the code. Without knowledge of what each variable means, it can be almost impossible to understand what the code is accomplishing. By adding well structured variable, function, and class names, code can be easier to understand. The method I was taught was that code should be able to be read by someone else without docstring or comments and they should still be able to understand the code. Implementing a variable list would help students better understand the code and allow students to be able to modify the code to better understand. Additionally, I would recommend adding type hints to parameters for functions as well as indicate what a function output is. This would improve clarity within the code as it would give students a direction for each function. Despite the suggestions, I felt the project was structured well and gave students a good understanding of the usefulness of engineering solvers.